

Volatile Keyword in C# with Examples

In this article, I am going to discuss the **Volatile Keyword in C#** with Examples. Please read our previous article where we discussed [Dynamic VS Reflection in C#](#) with Examples.

Volatile Keyword in C#

The Volatile Keyword in C# is one of the not discussed keywords. You can also say that not talked keyword or unknown keyword in C# language. More than 95% time, you will never use this keyword. But in case you are developing multi-threaded applications, and if you want to handle concurrency in a better manner, then you can use this volatile keyword.

According to MSDN, the volatile keyword indicates that a field might be modified by multiple threads that are executing at the same time. The compiler, the runtime system, and even the hardware may rearrange reads and writes to memory locations for performance reasons. Fields that are declared volatile are excluded from certain kinds of optimizations. Let us understand the need and use of the volatile keyword in C# with an example.

Example to Understand Volatile Keyword in C#:

In order to understand the Volatile Keyword in C#, what we will do is, first, we will discuss the problem we face due to concurrency issues in multi-threading applications. And then we will see how this volatile keyword helps us to solve the concurrency issues in multi-threading applications. For a better understanding of the concurrency issues, please have a look at the following sample code.

```
using System;
using System.Threading;
namespace VolatileKeywordDemo
{
    class Program
    {
        //Loop Variable
        private bool _loop = true;
        static void Main(string[] args)
        {
            //Calling the SomeMethod in a Multi-threaded manner
            Program obj1 = new Program();
            Thread thread1 = new Thread(SomeMethod);
            thread1.Start(obj1);
            //Pauses for 20 MS
            Thread.Sleep(20);
            //Setting the _loop value as false
            obj1._loop = false;
            Console.WriteLine("Step2:- _loop value set to False");
            Console.ReadKey();
        }
        //Simple Method
        public static void SomeMethod(object obj1)
        {
```

```

        Program obj = (Program)obj1;
        Console.WriteLine("Step1:- Entered into the Loop");
        while(obj._loop)
        {
        }
        Console.WriteLine("Step3:- Existed From the Loop");
    }
}
}

```

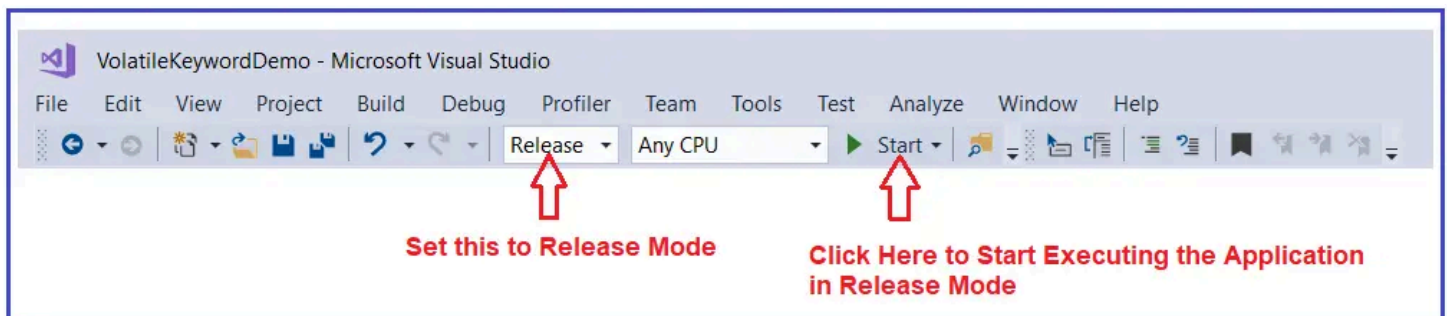
Here, first, we created a Boolean loop variable called `_loop` whose value is set to true. Then we created a simple method called `SomeMethod`. This `SomeMethod` method takes an object and that object is nothing but the `Program` class object, and hence we typecast the object to `Program` type and what this method does is, it will execute an infinite while loop i.e. until the loop variable `_loop` becomes false. By default, when the program is initializing, we set the `_loop` variable to true.

Then we call this `SomeMethod` in a multi-threaded manner from inside the `Main` method. So, what we do inside the `Main` method is, first we create an object of the `Program` class, then we created a thread instance and to the `Thread` constructor, we are passing the `SomeMethod` i.e. this thread we will execute the `SomeMethod` when we call the `Start` method. Further, you can notice to the `Start` method we are passing the `Program` class object. Once we call the `Start` method, the `SomeMethod` will start executing, and as part of the `SomeMethod`, the infinite while loop will execute.

Once the program starts executing the `SomeMethod`, the program halts for 20 Milliseconds. And after 20 seconds we are actually setting the `_loop` variable value to False. And here we expect that as soon as the `_loop` variable becomes false, the while loop which is executing inside the `SomeMethod` should be exited and the **Step3:- Existed From the Loop** statement should be printed in the console. This is because both are working on the same object and the object is by reference. So, we are expecting the output of the program to be as follows:

Step1:- Entered into the Loop
Step2:- `_loop` value set to False
Step3:- Existed From the Loop

Now, let us run the above code in Release mode and see the output. Why I am saying Release mode, which I will explain later part of this article. To run the application in Release mode, you need to set the execute option to Release in the Visual Studio Editor as shown in the below image.



Once you start executing the application in Release mode, you will get the following output.

```

Step1:- Entered into the Loop
Step2:- _loop value set to False

```

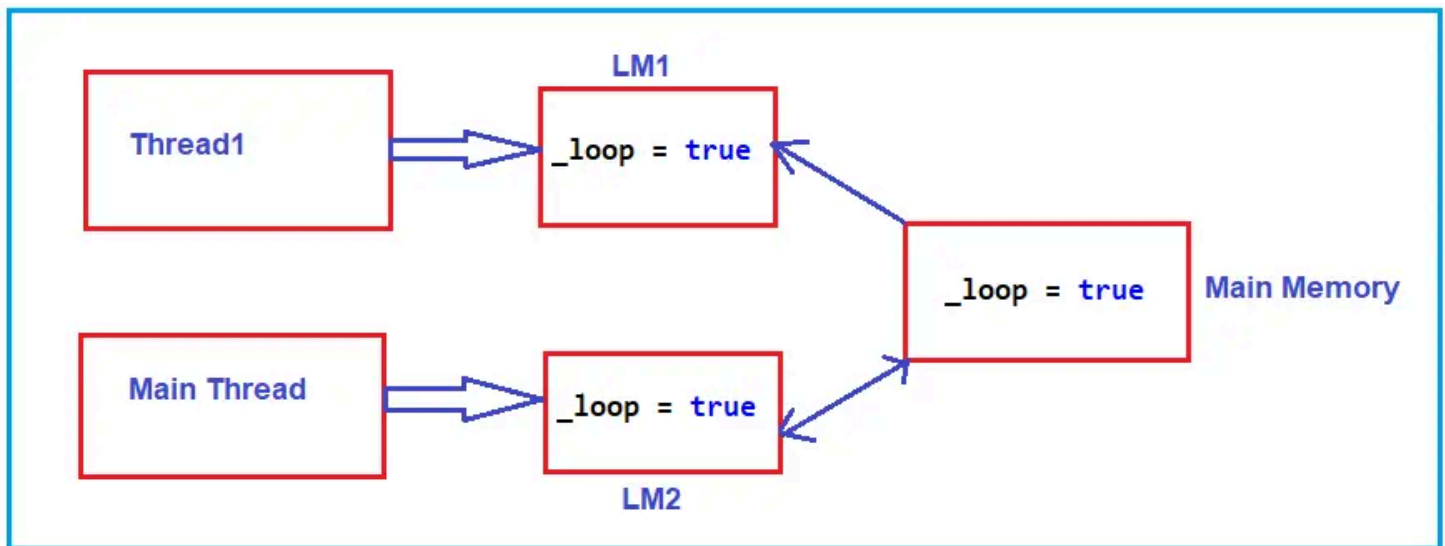
As you can see in the above output image, it will enter into the loop, after 20 milliseconds it will set the `_loop` variable value to false. But even after the loop value is set to False, the while loop is not exited. That means the thread (thread1) is still thinking that the `_loop` variable value is True. It means the value that we set inside the Main method (setting `_loop` variable to False) is not getting reflected inside the thread1 (i.e. inside the `SomeMethod`).

Why we are facing these concurrency Issues?

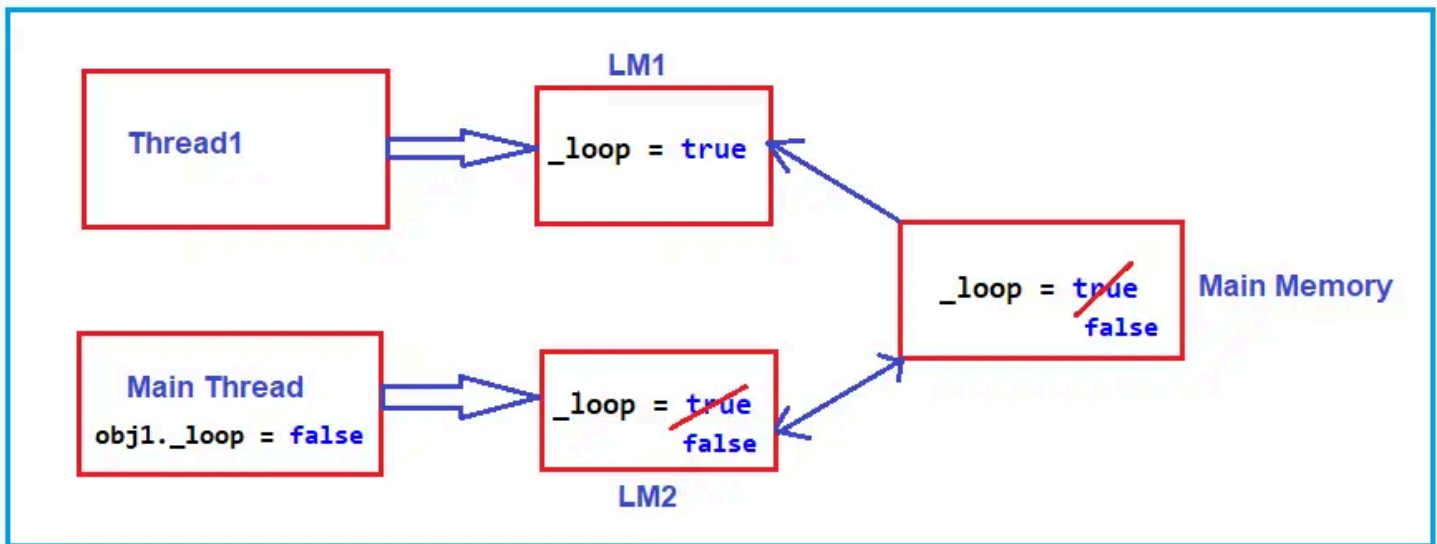
In order to understand why we are facing these concurrency issues, we need to understand the memory architecture of the above program. Please observe the following diagram. Here, we have two threads i.e. Main thread executing our application including the Main method, and thread2 executing the `SomeMethod`. And the variable `_loop` will be stored inside the Main memory and this variable is accessed by both threads. The Main memory will keep track of the `_loop` variable value. Here, the Main thread sets the `_loop` value to True. So, inside the Main memory, the `_loop` variable value will be True.

See, in order to improve the efficiency, these threads do not access the Main memory directly, rather they have their own local memory which is in sync with the main memory. Let us say the thread1 local memory is LM1 and the Main thread local memory is LM2. These local memories will have that loop variable. And there is a sync happening here and then between the main memory and the local memory of the threads.

No, Initially, when the program execution started, it set the `_loop` variable value to True. So, the `_loop` variable value will be true inside the Main memory, and also true inside the thread1 local memory as well as the Main thread local memory as shown in the below image.



Now, when the program running thread2 checks the `_loop` value and finds that value is true, so he will execute the while loop. After some time, the Main thread sets the `_loop` values to false. This will set the own local memory `_loop` value to false as well as the `_loop` value to false in the Main memory as shown in the below image.



As you can see the local memory of Thread1 is not updated. So, Thread1 still accessing the old value. The reason is that the local memory of Thread1 and Main memory, have not got a sync. Because of this reason, the updated data by the Main thread was not visible to Thread1.

How to Solve the above problem?

As the local memory and main memory do not in sync, we will get invalid results or unexpected results sometime. Now, the question is how to solve the above problem. What is the solution to this? How we can ensure that when Thread1 accesses the `_loop` variable (inside the local memory LM1), we need to go and sync up the `_loop` variable with the main memory? This is where we need to use the `volatile` keyword in C#.

Let us mark the `_loop` variable with the `volatile` keyword as shown in the below example.

```
using System;
using System.Threading;
namespace VolatileKeywordDemo
{
    class Program
    {
        //Loop Variable
        private volatile bool _loop = true;
        static void Main(string[] args)
        {
            //Calling the SomeMethod in a Multi-threaded manner
            Program obj1 = new Program();
            Thread thread1 = new Thread(SomeMethod);
            thread1.Start(obj1);
            //Pauses for 20 MS
            Thread.Sleep(20);
            //Setting the _loop value as false
            obj1._loop = false;
            Console.WriteLine("Step2:- _loop value set to False");
            Console.ReadKey();
        }
    }
}
```

```
//Simple Method
public static void SomeMethod(object obj1)
{
    Program obj = (Program)obj1;
    Console.WriteLine("Step1:- Entered into the Loop");
    while(obj._loop)
    {
    }
    Console.WriteLine("Step3:- Existed From the Loop");
}
}
```

So, when we mark the `_loop` variable as `volatile`, now what will happen is, whenever the while loop accesses this `_loop` variable, first, it will go and sync this local memory `_loop` variable data with the Main memory `_loop` variable data and then it will execute the loop. Now, if you execute the above code, you will get the output as expected as shown in the below image.

```
Step1:- Entered into the Loop
Step2:- _loop value set to False
Step3:- Existed From the Loop
```

So, you need to use the `volatile` keyword while you are doing multi-thread applications and especially when you accessing data that is concurrently updated by different threads and you want that updated data to be used by other threads. The `volatile` keyword ensures that the data you are accessing is up to date or you can say it is in sync with the main memory.

Note: Both in C# and Java, the keyword `volatile` tells the compiler that the value of the variable must never be cached as its value may change outside of the scope of the program itself. The compiler will then avoid any optimizations that may result in problems if the variable changes “outside of its control”.

Why do we run the application in Release mode?

See, by default, Debug includes debugging information in the compiled files (allowing easy debugging) while release usually has optimizations enabled. So, when you are developing an application, for easy debugging you need to use Debug. But, while deploying the application to the server, for better performance we need to publish the files in release mode.

I am Confused.

Even I bit confused about the `volatile` keyword and running the application using debug mode. If you run the above application in debug mode without using the `volatile` keyword, then you will get the output as expected. Let us modify the code as follows. Here, we are not using the `volatile` keyword.

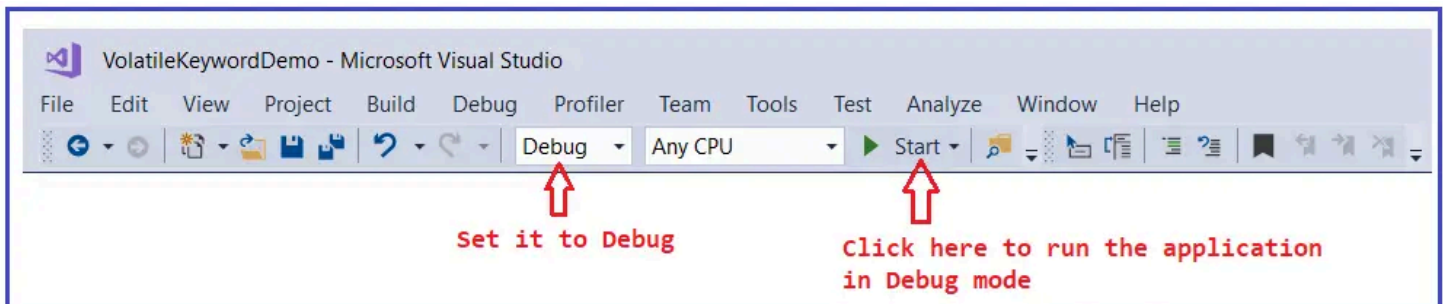
```
using System;
using System.Threading;
namespace VolatileKeywordDemo
{
    class Program
    {
```

```

//Loop Variable
private bool _loop = true;
static void Main(string[] args)
{
    //Calling the SomeMethod in a Multi-threaded manner
    Program obj1 = new Program();
    Thread thread1 = new Thread(SomeMethod);
    thread1.Start(obj1);
    //Pauses for 20 MS
    Thread.Sleep(20);
    //Setting the _loop value as false
    obj1._loop = false;
    Console.WriteLine("Step2:- _loop value set to False");
    Console.ReadKey();
}
//Simple Method
public static void SomeMethod(object obj1)
{
    Program obj = (Program)obj1;
    Console.WriteLine("Step1:- Entered into the Loop");
    while(obj._loop)
    {
    }
    Console.WriteLine("Step3:- Existed From the Loop");
}
}
}

```

Now, let us run the above code in Debug mode as shown in the below image.



Now, once you run the application in debug mode, you will get the following output.

```

Step1:- Entered into the Loop
Step2:- _loop value set to False
Step3:- Existed From the Loop

```

As you can see, here we are getting the output as expected. Still, I am searching for the reason why this is working fine in debug mode, but not working in Release mode. Once I found the reason, I will update it here. Meanwhile, if you found the reason why this is happening, please put the same in the comment section.

In the next article, I am going to discuss [Ref vs Out in C#](#) with Examples. Here, in this article, I try to explain the **Volatile Keyword in C#** with Examples. I hope you enjoy this Volatile Keyword in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.